

FINANCIAL MANAGEMENT SYSTEM – SQL PROJECT DOCUMENTATION

Prepared By: Shaik Fayaz Ahamed

Course: Data Analyst with GenAI Project

Duration: 7 Days

Financial Management System using SQL

This project is developed to understand the real-time usage of SQL in financial and banking systems. In this project, we created different tables such as Customers, Accounts, Transactions, Loans, and Employees. We performed operations like data insertion, data cleaning, joins, aggregate functions, TCL commands, DCL commands, and ALTER operations. This project helped us improve our practical SQL knowledge and understand how financial data is managed in databases.

This system ensures:

- Accurate financial record maintenance
- Secure transaction handling
- Proper customer account management
- Easy generation of financial reports
- Reliable database transaction control

Objectives of the Financial Management System

The primary objectives of the project are:

1. To design a relational database for financial operations.
2. To maintain customer and account details efficiently.
3. To store and manage transaction records securely.
4. To handle loans and employee information.
5. To perform data cleaning and validation.
6. To generate useful reports using SQL queries.
7. To implement transaction management using TCL commands.
8. To provide database security using DCL commands.
9. To understand real-time banking database operations.

MODULE - 1

Database Creation

In this module, we created the database and designed tables required for the Financial Management System. We created tables like Customers, Accounts, Transactions, Loans, and Employees with proper constraints and keys.

Customers Table

Column Name	Description
Customer_ID	Unique customer identification
Customer_Name	Name of customer
Phone_Number	Contact number
Email	Customer email address
City	Customer city
PAN_Number	Government identification

Accounts Table

Column Name	Description
Account_ID	Unique account number
Customer_ID	Links account to customer
Account_Type	Savings or Current
Balance	Available account balance
Open_Date	Date of account creation

Transactions Table

Column Name	Description
Transaction_ID	Unique transaction number
Account_ID	Account related to transaction
Transaction_Type	Deposit or Withdrawal
Amount	Transaction amount
Transaction_Date	Date of transaction

```
SQL> CREATE TABLE Customers (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL, Email VARCHAR(100), Phone VARCHAR(15), City VARCHAR(50), PAN_Number VARCHAR(20));
Table created.

SQL> CREATE TABLE Accounts (Account_ID INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHAR(50) NOT NULL, Balance DECIMAL(10,2), Open_Date DATE, FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID));
Table created.

SQL> CREATE TABLE Transactions (Transaction_ID INT PRIMARY KEY, Account_ID INT, Transaction_Type VARCHAR(20), Amount DECIMAL(10,2), Transaction_Date DATE, FOREIGN KEY (Account_ID) REFERENCES Accounts(Account_ID));
Table created.

SQL> CREATE TABLE Loans (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Amount DECIMAL(10,2), Loan_Type VARCHAR(50), Loan_Status VARCHAR(20), FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID));
Table created.

SQL> CREATE TABLE Employees (Employee_ID INT PRIMARY KEY, Employee_Name VARCHAR(100) NOT NULL, Department VARCHAR(50), Salary DECIMAL(10,2));
Table created.

SQL>
```

MODULE - 2

Data Modelling

In this module, we learned how relationships are created between tables using primary keys and foreign keys. We connected customers with accounts and accounts with transactions.

Relationship Between Customers and Accounts

One customer can have multiple accounts.

Relationship: One-to-Many Relationship

Example:

- One customer may own:
 - Savings account
 - Current account
 - Loan account

This relationship is implemented using Customer_ID as a foreign key in the Accounts table.

Relationship Between Accounts and Transactions

Each account can have multiple transactions.

Relationship: One-to-Many Relationship

Example:

- One account may have:
 - Deposits
 - Withdrawals
 - Online transfers

Account_ID acts as a foreign key in the Transactions table

\

Primary Keys

Primary keys uniquely identify records.

Table	Primary Key
Customers	Customer_ID
Accounts	Account_ID
Transactions	Transaction_ID
Loans	Loan_ID
Employees	Employee_ID

NOT NULL Columns

Some important columns should never contain NULL values.

MODULE - 3

Raw Data Insertion

In this module, we inserted customer records, account details, transaction records, employee details, and loan information. We also inserted duplicate and NULL values for practicing data cleaning operations.

Requirements

- Minimum 10 customers
- Minimum 10 accounts
- Minimum 20 transactions
- Minimum 5 employees

The inserted data intentionally includes:

- Duplicate values
- NULL values
- Negative balances
- Invalid transactions

Purpose

This helps students learn:

- Error handling
- Data cleaning
- Data validation
- Real-time database correction techniques

```
SQL> INSERT INTO Customers VALUES (1,'Rahul Sharma','rahul@gmail.com','9876543210','Hyderabad','ABCDE123HF');
1 row created.

SQL> INSERT INTO Customers VALUES (2,'Sneha Reddy',NULL,'9876500000','Vijayawada','PQRSX4567Y');
1 row created.

SQL> INSERT INTO Customers VALUES (3,'Amit Kumar','amit@gmail.com',NULL,'Guntur','LMNOP9876Z');
1 row created.

SQL> INSERT INTO Customers VALUES
INSERT INTO Customers VALUES
ERROR at line 1:
ORA-00926: missing expression

SQL> (4,'Priya Singh','priya@gmail.com','9999999999','Hyderabad','ASDFG123HH');
2 INSERT INTO Customers VALUES (4,'Priya Singh','priya@gmail.com','9999999999','Hyderabad','ASDFG123HH');
(4,'Priya Singh','priya@gmail.com','9999999999','Hyderabad','ASDFG123HH');
ERROR at line 1:
ORA-00928: missing SELECT keyword

SQL> INSERT INTO Customers VALUES (4,'Priya Singh','priya@gmail.com','9999999999','Hyderabad','ASDFG123HH');
1 row created.

SQL> INSERT INTO Customers VALUES (5,'Rahul Sharma','rahul@gmail.com','9876543210','Hyderabad','ABCDE123HF');
1 row created.

SQL> INSERT INTO Customers VALUES (6,'Kiran Rao','kiran@gmail.com','8888888888','Chennai','QWERT1234L');
1 row created.
```

```
SQL> INSERT INTO Employees VALUES
2 (1,'Arjun','Finance',50000);

1 row created.

SQL> INSERT INTO Employees VALUES (2,'Madhavi','HR',45000);

1 row created.

SQL> INSERT INTO Employees VALUES (3,'Suresh','IT',60000);

1 row created.

SQL> INSERT INTO Employees VALUES (4,'Ramesh','Finance',55000);

1 row created.

SQL> INSERT INTO Employees VALUES (5,'Kavya','Support',40000);

1 row created.
```

```
SQL> INSERT INTO Accounts VALUES (101,1,'Savings',50000,'2024-01-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (102,2,'Current',-2000,'2024-02-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (103,3,'Savings',70000,'2024-03-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (104,4,'Current',100000,'2024-01-15');
1 row created.

SQL> INSERT INTO Accounts VALUES (105,5,'Savings',30000,'2024-04-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (106,6,'Savings',-500,'2024-05-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (107,7,'Current',15000,'2024-06-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (108,8,'Savings',45000,'2024-07-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (109,9,'Current',80000,'2024-08-01');
1 row created.

SQL> INSERT INTO Accounts VALUES (110,10,'Savings',60000,'2024-09-01');
1 row created.

SQL> |
```

MODULE - 4

Data Cleaning

In this module, we identified NULL values, duplicate records, negative balances, and invalid transactions. We used UPDATE and DELETE commands to clean the data.

1. Finding NULL Email Addresses

The system identifies customers without email addresses.

```
SELECT * FROM Customers WHERE Email IS NULL;
```

```
SQL> select * from customers where email is NULL;
CUSTOMER_ID
-----
CUSTOMER_NAME
-----
EMAIL
-----
PHONE          CITY
-----
PAN_NUMBER
-----
                2
Sneha Reddy

CUSTOMER_ID
-----
CUSTOMER_NAME
-----
EMAIL
-----
PHONE          CITY
-----
PAN_NUMBER
-----
9876500000      Vijayawada
PQRSX4567Y
```

2. Replacing Missing Phone Number

NULL phone numbers are updated with default values.

```
UPDATE Customers SET Phone_Number='11111111' WHERE Phone_Number IS NULL;
```

```
SQL> update customers set phone=11111111 where phone is null;
2 rows updated.
SQL> select * from customers;
CUSTOMER_ID
-----
CUSTOMER_NAME
-----
EMAIL
-----
PHONE          CITY
-----
PAN_NUMBER
-----
                1
Rahul Sharma
rahul@gmail.com

CUSTOMER_ID
-----
CUSTOMER_NAME
-----
EMAIL
-----
PHONE          CITY
-----
PAN_NUMBER
-----
9876543210      Hyderabad
ABCDE1234F

CUSTOMER_ID
-----
CUSTOMER_NAME
-----
EMAIL
-----
```

3. Removing Negative Balances

Accounts with invalid negative balances are deleted or corrected.

DELETE FROM Accounts WHERE Balance < 0;

```
SQL> delete from accounts where balance<0;
delete from accounts where balance<0
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.SYS_C004046) violated - child record found

SQL> select * from accounts where balance<0;
ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE OPEN_DATE
-----
102 2 Current -2000 2024-02-01
106 6 Savings -500 2024-05-01

SQL> delete from transaction where account_id in(102,106);
delete from transaction where account_id in(102,106)
*
ERROR at line 1:
ORA-00942: table or view does not exist

SQL> delete from transactions where account_id in(102,106);
4 rows deleted.

SQL> delete from accounts where balance<0;
2 rows deleted.
```

4. Deleting Invalid Transactions

Transactions with invalid amounts are removed.

DELETE FROM Transactions WHERE Amount <= 0;

```
SQL> delete from accounts where balance<0;
delete from accounts where balance<0
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.SYS_C004046) violated - child record found

SQL> select * from accounts where balance<0;
ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE OPEN_DATE
-----
102 2 Current -2000 2024-02-01
106 6 Savings -500 2024-05-01

SQL> delete from transaction where account_id in(102,106);
delete from transaction where account_id in(102,106)
*
ERROR at line 1:
ORA-00942: table or view does not exist

SQL> delete from transactions where account_id in(102,106);
4 rows deleted.

SQL> delete from accounts where balance<0;
2 rows deleted.

SQL>
```

5. Identifying Duplicate Customers

Duplicate records are detected using SQL queries.

SELECT Customername, COUNT() FROM Customers GROUP BY Customer_name HAVING COUNT(*)>1;

```
Run SQL Command Line
SQL> SELECT CUSTOMER_NAME, COUNT(*) FROM CUSTOMERS GROUP BY CUSTOMER_NAME HAVING COUNT(*)>1;
CUSTOMER_NAME
-----
COUNT(*)
-----
Rahul Sharma
2
```

6. Standardizing City Names

City names are converted to uppercase.

UPDATE Customers SET City = UPPER(City);

```
SQL> UPDATE CUSTOMERS SET CITY=UPPER(CITY);
10 rows updated.

SQL> SELECT * FROM CUSTOMERS;

CUSTOMER_ID CUSTOMER_NAME                                EMAIL
PHONE      CITY
-----
1 Rahul Sharma                                rahul@gmail.com
9876543210 HYDERABAD
2 Sneha Reddy
9876500000 VIJAYAWADA
3 Amit Kumar                                amit@gmail.com
1111111111 GUNTUR
4 Priya Singh                                priya@gmail.com
9999999999 HYDERABAD
5 Rahul Sharma                                rahul@gmail.com
9876543210 HYDERABAD
6 Kiran Rao                                kiran@gmail.com
8888888888 CHENNAI
7 David                                david@gmail.com
7777777777 MUMBAI
8 Anjali                                anjali@gmail.com
1111111111 DELHI
9 Rakesh                                rakesh@gmail.com
6666666666 PUNE
10 Meena                                meena@gmail.com
5555555555 HYDERABAD

10 rows selected.
```


MODULE - 5

Data Manipulation

In this module, we updated balances, modified customer information, added new customers, and deleted inactive accounts. This module helped us understand real-time data management.

1. Updating Account Balances

Balances are updated after deposits or withdrawals.

```
UPDATE Accounts SET Balance = Balance+5000 Where Account_id=107;
```

2. Adding New Customers

New customers are inserted into the database.

```
INSERT INTO Customers VALUES (11,'Farhan','farhan@gmail.com','9871234567','Bangalore');
```

3. Deleting Inactive Accounts

Unused or inactive accounts are removed.

```
DELETE FROM Accounts WHERE Balance = 0;
```

4. Modifying Customer Details

Customer information can be updated when required.

Example

- Phone number changes
- Address updates
- Email modifications

```
UPDATE Customers SET Phone=7993738521 WHERE customer_id=4;
```

```
9 Rakesh
10 Meena
10 rows selected.
SQL> (11,'Farhan','farhan@gmail.com','9871234567','Bangalore');
(11,'Farhan','farhan@gmail.com','9871234567','Bangalore')
*
ERROR at line 1:
ORA-00928: missing SELECT keyword

SQL> insert into customers values(11,'farhan','fah@gmail.com',9668732578,'banglore');
1 row created.
SQL> delete from accounts where balance = 0;
0 rows deleted.
SQL> update customers set phone=7993738521 where customer_id=4;
1 row updated.
SQL> update loans set loan_amount= loan_amount+6000 where loan_status='Approved';
3 rows updated.
SQL> commit;
Commit complete.
SQL> |
```

MODULE - 6

SQL Queries & Reports

In this module, we used SELECT queries, JOIN queries, and aggregate functions. We generated reports such as total balance, average balance, and transaction history.

1. Display All Customers

SELECT * FROM Customers;

2. Show all account details

SELECT * FROM Accounts;

3. Find Customers from Hyderabad

SELECT * FROM Customers WHERE City='HYDERABAD';

4. Display Accounts with Balance Above 50,000

SELECT * FROM Accounts WHERE Balance > 50000;

```
SQL> select * from customers where city='HYDERABAD';
CUSTOMER_ID CUSTOMER_NAME          PHONE          CITY          EMAIL
-----
1 Rahul Sharma          9876543210     HYDERABAD     rahul@gmail.com
4 Priya Singh          7993738521     HYDERABAD     priya@gmail.com
5 Rahul Sharma          9876543210     HYDERABAD     rahul@gmail.com
10 Meena          5555555555     HYDERABAD     meena@gmail.com

SQL> select * from accounts where balance >50000;
ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE          BALANCE OPEN_DATE
-----
103          3 Savings          70000 2024-03-01
104          4 Current          105000 2024-01-15
109          9 Current          80000 2024-08-01
110         10 Savings          60000 2024-09-01

SQL> select c.customer_name,a.account_id,a.account_type,a.balance from customer c join account a on c.customer_id=a.customer_id;
select c.customer_name,a.account_id,a.account_type,a.balance from customer c join account a on c.customer_id=a.customer_id
*
ERROR at line 1:
ORA-00942: table or view does not exist

SQL> select c.customer_name,a.account_id,a.account_type,a.balance from customers c join accounts a on c.customer_id=a.customer_id;
CUSTOMER_NAME          ACCOUNT_ID ACCOUNT_TYPE          BALA
NCE
-----
Rahul Sharma          101 Savings          50
000
Amit Kumar          103 Savings          70
000
Priya Singh          104 Current          105
000
```

Join Queries

1. Customer Name with Account Details

Displays complete account information for each customer.

```
SELECT c.Customer_Name, a.Account_Type, a.Balance FROM Customers c JOIN Accounts a ON  
c.Customer_ID=a.Customer_ID;
```

2. Transaction History for Customers

Displays all the transaction history of all customers

```
SELECT c.Customer_Name, t.Transaction_Type, t.Amount FROM Customers c JOIN Accounts a ON  
c.Customer_ID=a.Customer_ID JOIN Transactions t ON a.Account_ID=t.Account_ID;
```

3. Find customers who made transactions above 10,000.

Displays all the transaction of customers who made transactions above 10,000.

```
SELECT DISTINCT C.Customer_Name FROM Customers C JOIN Accounts A ON C.Customer_ID =  
A.Customer_ID JOIN Transactions T ON A.Account_ID = T.Account_ID WHERE T.Amount > 10000;
```

4. Show account type-wise total balance.

Displays all the account type-wise total balance.

```
SQL> select c.customer_name, a.account_id,a.account_type,a.balance from customers c join accounts a on c.customer_id=a.customer_id;
```

CUSTOMER_NAME	ACCOUNT_ID	ACCOUNT_TYPE	BALANCE
Rahul Sharma	101	Savings	50000
Amit Kumar	103	Savings	70000
Priya Singh	104	Current	105000
Rahul Sharma	105	Savings	35000
David	107	Current	15000
Anjali	108	Savings	45000
Rakesh	109	Current	80000
Meena	110	Savings	60000

8 rows selected.

```
SQL> select c.customer_name,t.transaction_type,t.amount from customers c join accounts a on c.customer_id=a.customer_id join transaction t on a.account_id=t  
.account_id;  
select c.customer_name,t.transaction_type,t.amount from customers c join accounts a on c.customer_id=a.customer_id join transaction t on a.account_id=t.acco  
unt_id  
*  
ERROR at line 1:  
ORA-00942: table or view does not exist
```

```
SQL> select c.customer_name,t.transaction_type,t.amount from customers c join accounts a on c.customer_id=a.customer_id join transactions t on a.account_id=  
t.account_id;
```

CUSTOMER_NAME	TRANSACTION_TYPE	AMOUNT
Rahul Sharma	Deposit	10000
Rahul Sharma	Withdraw	5000
Amit Kumar	Withdraw	80000
Priya Singh	Deposit	15000
Rahul Sharma	Withdraw	500
David	Withdraw	2000
Anjali	Deposit	9000
Rakesh	Withdraw	1000
Meena	Deposit	5000
Rahul Sharma	Withdraw	1200

Aggregate Functions:

1. Find total bank balance.

```
SELECT Account_type, sum(Balance) as Total_balance FROM accounts GROUP BY Account_type;
```

2. Find average account balance.

```
SELECT AVG(Balance) as average_balance FROM Accounts;
```

3. Find maximum transaction amount.

```
SELECT MAX(Balance) as max_balance FROM Accounts;
```

4. Count total customers.

```
SELECT COUNT(*) AS total_customers FROM Customers
```

```
SQL> select distinct c.customer_name from customers c join accounts a on c.customer_id=a.customer_id join transactions t on a.account_id=t.account_id where t.amount>10000;

CUSTOMER_NAME
-----
Amit Kumar
Priya Singh

SQL> select account_type, sum(Balance) as Total_balance from accounts group by account_type;

ACCOUNT_TYPE          TOTAL_BALANCE
-----
Current                200000
Savings                260000

SQL> select sum(balance) as total_bank_balance from accounts;

TOTAL_BANK_BALANCE
-----
460000

SQL> select avg(balance) as average_balance from accounts;

AVERAGE_BALANCE
-----
57500

SQL> select avg(amount) as max_transaction from transactions;

MAX_TRANSACTION
-----
9743.75

SQL> select count(*) as total_customers from customers;

TOTAL_CUSTOMERS
-----
11
```

MODULE - 7

TCL Commands

In this module, we learned transaction management using COMMIT, ROLLBACK, and SAVEPOINT commands. These commands are important in banking systems for secure transactions.

1. Start a transaction for money transfer

START TRANSACTIONS;

2. Deduct amount from one account.

It Deduct amount from the account

UPDATE Accounts SET Balance=Balance-5000 WHERE Account_ID=101;

3. Add amount into another account.

It Add amount to the account

UPDATE Accounts SET Balance=Balance+5000 WHERE Account_ID=102;

4. Create SAVEPOINT.

SAVEPOINT transfer_point;

5. Rollback the transaction.

ROLLBACK TO transfer_point;

```
SQL> update accounts set balance=balance-5500 where account_id=103;
1 row updated.
SQL> update accounts set balance=balance+7400 where account_id=107;
1 row updated.
SQL> savepoint transfer_point;
Savepoint created.
SQL> rollback to transfer_point;
Rollback complete.
SQL> commit;
Commit complete.
SQL> |
```

MODULE - 8

DCL Commands

In this module, we learned about database security using GRANT and REVOKE commands. These commands are used to control access permissions for users.

1. Create a new database user.

```
CREATE USER finance_user IDENTIFIED BY finance123;
```

2. Grant SELECT and INSERT permissions.

```
GRANT SELECT, INSERT ON Customers TO finance_user;
```

3. Revoke DELETE permissions.

```
REVOKE SELECT ON Customers FROM finance_user;
```

4. Provide limited access to finance employees.

```
GRANT SELECT ON Customers TO finance_user;
```

```
SQL> create user finance_user identified by finance123;
User created.

SQL> grant select, insert on Financial_managemaent_system.* to finance_user;
grant select, insert on Financial_managemaent_system.* to finance_user
*
ERROR at line 1:
ORA-00903: invalid table name

SQL> grant select, insert on Customers to finance_user;
Grant succeeded.

SQL> revoke delete on customers to finance_user;
revoke delete on customers to finance_user
*
ERROR at line 1:
ORA-00905: missing keyword

SQL> revoke delete on customers from finance_user;
revoke delete on customers from finance_user
*
ERROR at line 1:
ORA-01927: cannot REVOKE privileges you did not grant

SQL> revoke select , insert on customers from finance_user;
Revoke succeeded.

SQL> grant select on employees to finace_user;
grant select on employees to finance_user
*
ERROR at line 1:
ORA-01917: user or role 'FINACE_USER' does not exist

SQL> grant select on employees to finance_user;
Grant succeeded.
```

MODULE - 9

ALTER & DROP Operations

In this module, we used ALTER TABLE commands to add and rename columns and DROP commands to remove unnecessary tables.

1. Add PAN Number column to Customers table.

```
ALTER TABLE Customers ADD PAN_Number VARCHAR2(20);
```

2. Add Loan Proof column.

```
ALTER TABLE Loans ADD Loan_Status VARCHAR2(20);
```

3. Rename a column.

```
ALTER TABLE Customers RENAME COLUMN Phone TO Contact_No;
```

```
SQL> alter table customers add (pan_number varchar2(15));
Table altered.

SQL> alter table loans add (loan_status varchar2(15));
alter table loans add (loan_status varchar2(15))
*
ERROR at line 1:
ORA-01430: column being added already exists in table

SQL> desc loans;
Name                                         Null?    Type
-----
LOAN_ID                                     NOT NULL NUMBER(38)
CUSTOMER_ID                                NUMBER(38)
LOAN_AMOUNT                                NUMBER(18,2)
LOAN_TYPE                                  VARCHAR2(50)
LOAN_STATUS                                VARCHAR2(20)

SQL> alter table loans add (Loan_proof varchar2(15));
Table altered.

SQL> alter table customers rename column phone to contact_no;
Table altered.

SQL> drop table employees;
Table dropped.

SQL> |
```

Conclusion

The Financial Management System project provides practical knowledge of SQL database operations used in real-world banking systems. It demonstrates how financial information can be stored, cleaned, manipulated, secured, and analyzed efficiently using relational databases.

Through this project, students gain hands-on experience with:

- Database design
- SQL queries
- Data cleaning
- Joins and relationships
- TCL and DCL commands
- Financial reporting
- Transaction management

This project improves understanding of real-time financial database systems and prepares students for industry-level database development and management tasks.

Advantages of Financial Management System

1. Easy management of customer accounts
2. Secure transaction handling
3. Faster report generation
4. Improved data consistency
5. Reduced manual work
6. Better financial tracking
7. Enhanced database security
8. Efficient loan management

